ENGINEERING & ARCHITECTURE SPECIFICATION

# Hostel & PG Local Ingestion Agent

Production Implementation Guide, Relational Database Mapping, AES Binary Image Encryption, and Complete Source Code for MySQL 8.0+.

SYSTEM TYPE

**Automated Batch Ingestion Pipeline**

SECURITY PROTOCOL

**Fernet AES-128-CBC + HMAC SHA-256**

TARGET DATABASE

**MySQL 8.0+ (InnoDB)**

IMPLEMENTATION LANGUAGE

**Python 3.8+ (PyMySQL, Cryptography)**

## 1. Executive Overview & Pipeline Objectives

The **Hostel & PG Local Ingestion Agent** provides a resilient, autonomous pipeline designed to parse tabular accommodation directories (such as CSV datasets) and safely ingest records into a normalized MySQL relational schema.

Unlike naive import utilities, this agent handles complex data relationships across four relational entities, enforces ACID transaction safety per record, automatically downloads external property room photos, applies military-grade symmetric encryption to the binary payloads, and provides a dual-tiered error recovery workflow (exporting failed records to both CSV and multi-sheet Excel workbooks).

## 2. Relational Database Architecture & ER Model

The database schema distributes accommodation data across four interdependent tables to maintain first normal form (1NF) and third normal form (3NF) compliance:




### 2.1 Table Schema Specifications

**Address Table (address)**

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| **id** | bigint | NO | PRI | NULL | Primary Key (auto_increment) |
| **city** | varchar(255) | YES | | NULL | Metropolitan city (e.g. Hyderabad) |
| **country** | varchar(255) | YES | | NULL | Country name (India) |
| **state** | varchar(255) | YES | | NULL | State jurisdiction (Telangana) |
| **street** | varchar(255) | YES | | NULL | Arterial road / street name |
| **zipcode** | varchar(255) | YES | | NULL | Postal / PIN code (e.g. 500032) |
| **version** | bigint | YES | | 0 | Optimistic locking version |
| **area** | varchar(255) | YES | | NULL | Neighborhood or sub-locality |
| **house_no** | varchar(255) | YES | | NULL | Door / plot / building number |
| **town** | varchar(255) | YES | | NULL | Municipal town or circle |

**Hostel Table (hostel)**

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| **id** | bigint | NO | PRI | NULL | Primary Key (auto_increment) |
| **is_active** | bit(1) | NO | | b'1' | Status flag (Set to True / 1) |

| Field | Type | Null | Key | Default | Description |
|---|---|---|---|---|---|
| **is_for_sale** | bit(1) | NO | | b'0' | Sale flag (Set to False / 0) |
| **address_id** | bigint | YES | UNI | NULL | Unique Foreign Key referencing address.id |
| **group_id** | bigint | YES | MUL | 10 | Tenant / Org Group ID (Set to 10) |
| **contact** | varchar(255) | YES | | NULL | Contact phone number |
| **name** | varchar(255) | YES | | NULL | Commercial hostel / PG name |
| **type** | varchar(255) | YES | | NULL | Category: MEN, WOMEN, or COLIVE |
| **purpose** | varchar(255) | YES | | 'COMMERCIAL' | Usage purpose (Set to COMMERCIAL) |
| **rooms** | bigint | YES | | 0 | Total registered rooms (Set to 0) |
| **version** | bigint | YES | | 0 | Optimistic locking version (0) |
| **created_by** | varchar(255) | NO | | '1' | User ID of creator (Set to '1') |
| **last_modified_by** | varchar(255) | YES | | '1' | User ID of editor (Set to '1') |
| **last_modified_date** | datetime(6) | YES | | NOW | Timestamp with microsecond precision |
| **created_date** | datetime(6) | YES | | NOW | Timestamp with microsecond precision |
| **price_ends** | double | NO | | 0.0 | Upper rent boundary (INR) |
| **price_starts** | double | NO | | 0.0 | Lower rent boundary (INR) |
| **location** | varchar(255) | YES | | NULL | Full formatted address string |

**Image IDs Table (image_ids)**

| Field | Type | Null | Key | Description |
|---|---|---|---|---|
| **id** | bigint | NO | PRI | Primary Key (auto_increment) |
| **relate_id** | bigint | YES | MUL | Entity identifier referencing hostel.id |
| **relate_to** | varchar(50) | YES | | Polymorphic type ('HOSTEL_IMAGE') |
| **image_name** | varchar(255) | YES | | Clean file name (e.g. room_photo_1.jpg) |

**Images Table (images)**

| Field | Type | Null | Key | Description |
|---|---|---|---|---|
| **id** | int | NO | PRI | Primary Key (auto_increment) |
| **image_id_id** | bigint | YES | MUL | Foreign key referencing image_ids.id |
| **filename** | varchar(255) | YES | | Extracted disk or asset filename |
| **type** | varchar(255) | YES | | MIME type (e.g. image/jpeg, image/png) |
| **url** | varchar(255) | YES | | Source HTTP/HTTPS image URL |
| **data** | longblob | YES | | **Encrypted binary payload (AES-128)** |
| **hostel_id** | bigint | YES | MUL | Foreign key referencing hostel.id |
| **uploaded_date** | datetime | YES | | Upload timestamp |
| **image_name** | varchar(255) | NO | | Clean image label / display name |
| **related_id** | bigint | NO | | Foreign entity ID (matches hostel_id) |
| **relate_to** | enum(...) | NO | | Fixed ENUM value: 'HOSTEL_IMAGE' |
| **name** | varchar(255) | YES | | Hostel / property name label |

## 3. Security Architecture: Image Encryption Workflow

In modern web architectures, directly storing external image URLs can lead to broken references, rate limiting, and unauthorized hotlinking. Furthermore, storing raw unencrypted photos in the database can expose user and property visual data. The agent implements **Fernet AES-128-CBC encryption with HMAC SHA-256 integrity verification**.

### 3.1 Encryption Flow Diagram

```
[ image_url ]
      |
      v (HTTP GET Stream via urllib with User-Agent)
[ Raw Binary Bytes (JPEG/PNG) ]
      |
      v (ImageEncryptor.encrypt() using Fernet Secret Key)
[ AES-128 Encrypted Bytes ]
      |
      v (SQL INSERT INTO images.data)
[ MySQL LONGBLOB Storage ]
```

### 3.2 Application UI Decryption Snippet

To retrieve and display the encrypted image within your application UI (such as a Flask/FastAPI backend or desktop client), use the following Python decryption pattern:

```
import os
from cryptography.fernet import Fernet

# Initialize cipher with the shared application key
cipher = Fernet(os.environ["IMAGE_ENCRYPTION_KEY"].encode("utf-8"))

# Fetch encrypted BLOB from images table
cursor.execute("SELECT data, type, filename FROM images WHERE hostel_id = %s", (hostel_id,))
record = cursor.fetchone()

# Decrypt binary payload back to native image format
decrypted_image_bytes = cipher.decrypt(record["data"])

# Serve as HTTP response or write to cached media directory
# e.g., Response(decrypted_image_bytes, mimetype=record["type"])
```

## 4. Transaction Isolation & Error Recovery

To guarantee strict database consistency and eliminate orphaned records, the ingestion agent wraps the operations for each individual hostel record inside an isolated database transaction:

- **Atomic Rollback ( `conn.rollback()` )**: If an image fails to download, a database connection experiences a glitch, or a duplicate key conflict arises, the agent rolls back the entire transaction for that record. The `address`, `hostel`, `image_ids`, and `images` tables remain completely untainted.
- **Multi-Format Error Reporting**: When one or more records fail, the agent captures the exact error string and generates:
  1. `<filename>_failed_records.csv`: Contains all original CSV fields plus `failure_reason` and `failed_at` timestamp.
  2. `<filename>_processed_with_failures.xlsx`: A multi-sheet Excel report containing an `Original_Data` worksheet alongside a dedicated `Failed_Records` worksheet.

## 5. Complete MySQL Database DDL Schema (schema.sql)

The complete, executable DDL migration script to initialize the required tables, keys, and foreign constraints in MySQL 8.0+:

```
-- ========================================================================
-- Database Schema for Hostel & PG Management System
-- Tables: address, hostel, image_ids, images
-- ========================================================================

CREATE DATABASE IF NOT EXISTS hostel_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE hostel_db;

-- 1. Address Table
CREATE TABLE IF NOT EXISTS address (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    city VARCHAR(255) NULL,
    country VARCHAR(255) NULL,
    state VARCHAR(255) NULL,
    street VARCHAR(255) NULL,
    zipcode VARCHAR(255) NULL,
    version BIGINT NULL DEFAULT 0,
    area VARCHAR(255) NULL,
    house_no VARCHAR(255) NULL,
    town VARCHAR(255) NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. Hostel Table
CREATE TABLE IF NOT EXISTS hostel (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    is_active BIT(1) NOT NULL DEFAULT b'1',
    is_for_sale BIT(1) NOT NULL DEFAULT b'0',
    address_id BIGINT NULL UNIQUE,
    group_id BIGINT NULL,
    contact VARCHAR(255) NULL,
    name VARCHAR(255) NULL,
    type VARCHAR(255) NULL,
```

```
    purpose VARCHAR(255) NULL DEFAULT 'COMMERCIAL',
    rooms BIGINT NULL DEFAULT 0,
    version BIGINT NULL DEFAULT 0,
    created_by VARCHAR(255) NOT NULL DEFAULT '1',
    last_modified_by VARCHAR(255) NULL DEFAULT '1',
    last_modified_date DATETIME(6) NULL,
    created_date DATETIME(6) NULL,
    price_ends DOUBLE NOT NULL,
    price_starts DOUBLE NOT NULL,
    location VARCHAR(255) NULL,
    INDEX idx_hostel_group_id (group_id),
    CONSTRAINT fk_hostel_address FOREIGN KEY (address_id) REFERENCES address (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. Image IDs Table
CREATE TABLE IF NOT EXISTS image_ids (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    relate_id BIGINT NULL,
    relate_to VARCHAR(50) NULL,
    image_name VARCHAR(255) NULL,
    INDEX idx_image_ids_relate (relate_id, relate_to)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. Images Table
CREATE TABLE IF NOT EXISTS images (
    id INT AUTO_INCREMENT PRIMARY KEY,
    image_id_id BIGINT NULL,
    filename VARCHAR(255) NULL,
    type VARCHAR(255) NULL,
    url VARCHAR(255) NULL,
    data LONGBLOB NULL,
    hostel_id BIGINT NULL,
    uploaded_date DATETIME NULL,
    image_name VARCHAR(255) NOT NULL,
    related_id BIGINT NOT NULL,
    relate_to ENUM('EXPENSE_IMAGE','HOSTEL_IMAGE','TENANT_IMAGE','USER_IMAGE') NOT NULL,
    name VARCHAR(255) NULL,
    INDEX idx_images_hostel_id (hostel_id),
    INDEX idx_images_image_id_id (image_id_id),
    INDEX idx_images_related (related_id, relate_to),
    CONSTRAINT fk_images_hostel FOREIGN KEY (hostel_id) REFERENCES hostel (id) ON DELETE CASCADE,
    CONSTRAINT fk_images_image_ids FOREIGN KEY (image_id_id) REFERENCES image_ids (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

## 6. Complete Production Agent Source Code (hostel_db_agent.py)

The complete, unedited Python source code for the automated ingestion pipeline:

```
 #!/usr/bin/env python3
"""
Hostel & PG MySQL Ingestion Agent
=================================
Automated pipeline to process hostel/PG directory CSV files, download and encrypt
property images, and store records across normalized relational MySQL tables:
  1. address
  2. hostel (FK -> address.id)
  3. image_ids (relate_id -> hostel.id)
  4. images (FK -> hostel.id, FK -> image_ids.id)

Includes transactional rollbacks per record and writes failed records to both
a 'failed_records.csv' and a multi-sheet Excel report ('processed_with_failures.xlsx').
"""

import os
import sys
import csv
import logging
import argparse
import datetime
import mimetypes
import urllib.request
import urllib.error
from urllib.parse import urlparse
from typing import Dict, Any, List, Optional, Tuple

try:
    import pymysql
    from pymysql.cursors import DictCursor
except ImportError:
    pymysql = None

try:
    import pandas as pd
except ImportError:
    pd = None

try:
    from cryptography.fernet import Fernet
except ImportError:
    Fernet = None
```

```
# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)]
)
logger = logging.getLogger("HostelDBAgent")


class ImageEncryptor:
    """Handles symmetric AES (Fernet) encryption for binary image data."""

    def __init__(self, key: Optional[bytes] = None):
        if Fernet is None:
            raise ImportError("Package 'cryptography' is required. Install via: pip install cryptography")

        if key is None:
            env_key = os.environ.get("IMAGE_ENCRYPTION_KEY")
            if env_key:
                self.key = env_key.encode("utf-8")
            else:
                self.key = Fernet.generate_key()
                logger.info(f"Generated new encryption key: {self.key.decode('utf-8')}")
                logger.info("Store this in your environment as IMAGE_ENCRYPTION_KEY to decrypt in your application.")
        else:
            self.key = key

        self.cipher = Fernet(self.key)

    def encrypt(self, raw_bytes: bytes) -> bytes:
        """Encrypt binary image data."""
        return self.cipher.encrypt(raw_bytes)

    def decrypt(self, encrypted_bytes: bytes) -> bytes:
        """Decrypt binary image data."""
        return self.cipher.decrypt(encrypted_bytes)


class HostelIngestionAgent:
    """Processes CSV records and safely inserts into MySQL database."""

    def __init__(self, db_config: Dict[str, Any], encryptor: ImageEncryptor):
        if pymysql is None:
            raise ImportError("Package 'pymysql' is required. Install via: pip install pymysql")
        self.db_config = db_config
        self.encryptor = encryptor

    def get_connection(self):
        """Establish a MySQL connection."""
        return pymysql.connect(
            host=self.db_config.get("host", "localhost"),
            port=int(self.db_config.get("port", 3306)),
            user=self.db_config.get("user", "root"),
            password=self.db_config.get("password", ""),
            database=self.db_config.get("database", "hostel_db"),
            charset="utf8mb4",
            autocommit=False,
            cursorclass=DictCursor
        )

    def download_image(self, url: str, timeout: int = 10) -> Tuple[bytes, str, str]:
        """
        Download image from URL, returning (raw_bytes, mime_type, filename).
        Fallback to placeholder bytes if URL download fails.
        """
        parsed = urlparse(url)
        path = parsed.path
        filename = os.path.basename(path) or "hostel_image.jpg"
        if not os.path.splitext(filename)[1]:
            filename += ".jpg"

        mime_type, _ = mimetypes.guess_type(filename)
        if not mime_type:
            mime_type = "image/jpeg"

        req = urllib.request.Request(
            url,
            headers={"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"}
        )

        with urllib.request.urlopen(req, timeout=timeout) as resp:
            content_type = resp.headers.get_content_type()
            if content_type and "image" in content_type:
                mime_type = content_type
            raw_bytes = resp.read()

        return raw_bytes, mime_type, filename

    def insert_record(self, cursor, row: Dict[str, Any]) -> int:
        """
        Execute atomic insert across Address, Hostel, Image_IDs, and Images tables.
        Returns newly created hostel_id.
```

```
        """
        now = datetime.datetime.now()

        # 1. Insert Address
        address_sql = """
            INSERT INTO address (
                city, country, state, street, zipcode, version, area, house_no, town
            ) VALUES (
                %s, %s, %s, %s, %s, %s, %s, %s, %s
            )
        """
        address_values = (
            str(row.get("city", ""))[:255] or None,
            str(row.get("country", "India"))[:255] or None,
            str(row.get("state", ""))[:255] or None,
            str(row.get("street", ""))[:255] or None,
            str(row.get("pincode", row.get("zipcode", "")))[:255] or None,
            0,  # version
            str(row.get("area", ""))[:255] or None,
            str(row.get("house_no", ""))[:255] or None,
            str(row.get("town", row.get("city", "")))[:255] or None,
        )
        cursor.execute(address_sql, address_values)
        address_id = cursor.lastrowid

        # 2. Insert Hostel
        p_starts = float(row.get("price_starts", 0.0) or 0.0)
        p_ends = float(row.get("price_ends", 0.0) or 0.0)

        hostel_sql = """
            INSERT INTO hostel (
                is_active, is_for_sale, address_id, group_id, contact, name,
                type, purpose, rooms, version, created_by, last_modified_by,
                last_modified_date, created_date, price_ends, price_starts, location
            ) VALUES (
                %s, %s, %s, %s, %s, %s,
                %s, %s, %s, %s, %s, %s,
                %s, %s, %s, %s, %s
            )
        """
        hostel_values = (
            1,  # is_active: True
            0,  # is_for_sale: False
            address_id,
            10,  # group_id: 10
            str(row.get("contact", ""))[:255] or None,
            str(row.get("name", ""))[:255] or None,
            str(row.get("type", ""))[:255] or None,
            "COMMERCIAL",  # purpose: COMMERCIAL
            0,  # rooms: 0
            0,  # version: 0
            "1",  # created_by: 1
            "1",  # last_modified_by: 1
            now,
            now,
            p_ends,
            p_starts,
            str(row.get("location", ""))[:255] or None,
        )
        cursor.execute(hostel_sql, hostel_values)
        hostel_id = cursor.lastrowid

        # 3. Handle Image Download & Encryption
        image_url = row.get("image_url", "").strip()
        if image_url:
            raw_bytes, mime_type, filename = self.download_image(image_url)
            encrypted_data = self.encryptor.encrypt(raw_bytes)

            # Insert Image_IDs
            image_ids_sql = """
                INSERT INTO image_ids (
                    relate_id, relate_to, image_name
                ) VALUES (
                    %s, %s, %s
                )
            """
            cursor.execute(image_ids_sql, (hostel_id, "HOSTEL_IMAGE", filename[:255]))
            image_id_val = cursor.lastrowid

            # Insert Images table
            images_sql = """
                INSERT INTO images (
                    image_id_id, filename, type, url, data, hostel_id,
                    uploaded_date, image_name, related_id, relate_to, name
                ) VALUES (
                    %s, %s, %s, %s, %s, %s,
                    %s, %s, %s, %s, %s
                )
            """
            images_values = (
                image_id_val,
                filename[:255],
                mime_type[:255],
```

```
                image_url[:255],
                encrypted_data,
                hostel_id,
                now,
                filename[:255],
                hostel_id,
                "HOSTEL_IMAGE",
                str(row.get("name", ""))[:255]
            )
            cursor.execute(images_sql, images_values)

        return hostel_id

    def process_file(self, csv_path: str, failed_output_path: Optional[str] = None) -> Dict[str, Any]:
        """
        Reads CSV file, processes records row-by-row with transactional isolation,
        and saves any failed records to a failure report.
        """
        if not os.path.exists(csv_path):
            raise FileNotFoundError(f"Input file not found at: {csv_path}")

        records = []
        with open(csv_path, mode="r", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                records.append(row)

        total_count = len(records)
        success_count = 0
        failed_records: List[Dict[str, Any]] = []

        logger.info(f"Loaded {total_count} records from {csv_path}. Initiating database ingestion...")

        conn = self.get_connection()
        try:
            for idx, row in enumerate(records, start=1):
                try:
                    with conn.cursor() as cursor:
                        hostel_id = self.insert_record(cursor, row)
                    conn.commit()
                    success_count += 1
                    if idx % 50 == 0 or idx == total_count:
                        logger.info(f"Progress: [{idx}/{total_count}] records processed successfully.")
                except Exception as err:
                    conn.rollback()
                    logger.warning(f"Row {idx} failed ('{row.get('name', 'Unknown')}'): {err}")
                    failed_row = dict(row)
                    failed_row["failure_reason"] = str(err)
                    failed_row["failed_at"] = datetime.datetime.now().isoformat()
                    failed_records.append(failed_row)
        finally:
            conn.close()

        # Handle Failed Records Output
        if failed_records:
            base_dir = os.path.dirname(csv_path) or "."
            stem = os.path.splitext(os.path.basename(csv_path))[0]

            # 1. Output CSV failed records
            csv_fail_path = failed_output_path or os.path.join(base_dir, f"{stem}_failed_records.csv")
            fieldnames = list(records[0].keys()) + ["failure_reason", "failed_at"]
            with open(csv_fail_path, mode="w", newline="", encoding="utf-8") as f:
                writer = csv.DictWriter(f, fieldnames=fieldnames)
                writer.writeheader()
                writer.writerows(failed_records)
            logger.info(f"Exported {len(failed_records)} failed records to CSV: {csv_fail_path}")

            # 2. Output multi-sheet Excel file if pandas/openpyxl is available
            excel_path = os.path.join(base_dir, f"{stem}_processed_with_failures.xlsx")
            if pd is not None:
                try:
                    with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
                        pd.DataFrame(records).to_excel(writer, sheet_name="Original_Data", index=False)
                        pd.DataFrame(failed_records).to_excel(writer, sheet_name="Failed_Records", index=False)
                    logger.info(f"Exported multi-sheet report with Failed_Records to: {excel_path}")
                except Exception as ex:
                    logger.warning(f"Could not generate Excel report: {ex}")

        summary = {
            "total_records": total_count,
            "success_count": success_count,
            "failed_count": len(failed_records),
            "status": "COMPLETED" if len(failed_records) == 0 else "PARTIAL_SUCCESS"
        }
        logger.info(f"Ingestion finished: {summary}")
        return summary


def main():
    parser = argparse.ArgumentParser(description="Hostel & PG MySQL Database Ingestion Agent")
    parser.add_argument("--csv", required=True, help="Path to input CSV file")
    parser.add_argument("--host", default="localhost", help="MySQL Host (default: localhost)")
    parser.add_argument("--port", type=int, default=3306, help="MySQL Port (default: 3306)")
```

```
    parser.add_argument("--user", default="root", help="MySQL Username (default: root)")
    parser.add_argument("--password", default="", help="MySQL Password")
    parser.add_argument("--db", default="hostel_db", help="MySQL Database name (default: hostel_db)")
    parser.add_argument("--key", default=None, help="Fernet AES 32-byte Base64 encryption key (optional)")

    args = parser.parse_args()

    db_config = {
        "host": args.host,
        "port": args.port,
        "user": args.user,
        "password": args.password,
        "database": args.db
    }

    encryptor = ImageEncryptor(key=args.key.encode("utf-8") if args.key else None)
    agent = HostelIngestionAgent(db_config, encryptor)
    agent.process_file(args.csv)


if __name__ == "__main__":
    main()
```

## 7. Local Execution & Runbook Guide

### 7.1 Prerequisites & Installation

Install the required Python packages into your local environment:

```
pip install pymysql cryptography pandas openpyxl
```

### 7.2 Database Initialization

Execute the schema file against your local MySQL server:

```
mysql -u root -p < schema.sql
```

### 7.3 Executing the Pipeline

Run the agent against your dataset:

```
python hostel_db_agent.py   --csv /path/to/hostels_pg_directory_india.csv   --host localhost   --port 3306   --user root   --password
your_secure_password   --db hostel_db
```

### 7.4 Common Operational Scenarios & Troubleshooting

| Issue / Symptom | Root Cause | Recommended Solution |
|---|---|---|
| **Packet Too Large Error** ( `PacketSequenceNumberException` ) | Default MySQL `max_allowed_packet` is smaller than high-resolution encrypted image BLOBs. | Update `my.cnf` or execute: `SET GLOBAL max_allowed_packet=67108864;` (64MB) and restart connection. |
| **Invalid Key Token Error** during UI image decryption | Mismatch between encryption key used at insertion time and UI application environment variable. | Ensure `IMAGE_ENCRYPTION_KEY` is permanently stored in your secret manager or application `.env` file. |
| **Foreign Key Constraint Violation** on Address or Hostel | Address insertion failed or was rolled back prior to referencing. | The agent uses atomic per-row commits; review `failed_records.csv` for specific column truncation or type errors. |
| **HTTP Timeout on Image Retrieval** | External image host latency or network firewall blocks URL. | Adjust `timeout` in `download_image()` or supply pre-cached local image file URIs in CSV. |

> **Security Recommendation:** In production environments, avoid passing database passwords directly as CLI parameters. Instead, configure environment variables or utilize a local `.env` file.

# IMPLEMENTATION UPDATE

Hostel & PG Local Ingestion Agent | Current behavior

*Revision date: 15 October 2026*

## Purpose and precedence

This update records the behavior implemented in hostel-csv-parser.py after the original specification was produced. Where this update conflicts with the earlier schema examples, command-line instructions, or embedded source excerpts, this update describes the current script and takes precedence. The database schema is not changed by the script.

## 1. Running the importer and reading the CSV

- Run the Python file directly; the current entry point does not require command-line arguments.
- The input file is hostels-india-data.csv in the same directory as the Python script. The path is resolved from the script location, not the current working directory.
- CSV input is read as UTF-8 with optional BOM support. Rows with a different number of fields than the header are rejected with a row-specific error.
- Repeated header names are preserved by suffixing later occurrences. For this data file, the two location columns are read as location and location_2; the importer prefers location_2 for hostel.location.

## 2. Contact normalization and duplicate handling

- Contact values are reduced to digits. A 12-digit value starting with 91 has that country prefix removed, yielding the 10-digit Indian contact form.
- Before downloading an image or inserting any row data, the importer checks hostel.contact for the normalized value and supported legacy forms (+91 12345 67890 and 12345 67890).
- A matching contact causes the entire CSV row to be skipped: no address, hostel, amenity link/master, image ID, or image record is inserted, and no image is downloaded.
- The summary reports total_records, success_count, skipped_count, failed_count, and status. A duplicate increments skipped_count, not failed_count.

# 3. Database writes and transaction behavior

For a non-duplicate row, database writes run in one per-row transaction. On success the connection commits; on an exception it rolls back and records the source row plus failure details in the failed-record report.

| Table | Current insert behavior |
|---|---|
| address | Insert CSV address fields; set version to 0; use the generated address.id. |
| hostel | Insert the hostel using address.id, group_id 10, normalized contact, CSV name/type/rent/location, and the current fixed status/purpose/creator values. The schema requires created_by, price_starts, and price_ends; the importer supplies them. |
| amenities | Split the CSV amenities cell on commas, trim names, and deduplicate repeated names in a row. Look up a case-insensitive name; insert a missing master with is_available=1. |
| hostel_amenities | Insert one (amenity_id, hostel_id) link per distinct amenity for the new hostel. |
| image_ids and images | Only for a successfully downloaded, non-empty image response with image/* content type. Store encrypted bytes in images.data. |

### MySQL constraints that affect inserts

- The current database has an existing partner_group with id 10, referenced by hostel.group_id.
- The deployed image_ids.id column is a required primary key and is not AUTO_INCREMENT. The importer therefore assigns COALESCE(MAX(id), 0) + 1 and explicitly inserts that id.
- A MySQL named lock (hostel-csv-parser-ingestion) serializes duplicate checks, amenity creation, and image_ids ID allocation among importer runs. The lock is released after each row is committed or rolled back.
- The amenities table has no unique index on name in the inspected schema. The importer performs a case-insensitive lookup while holding its named lock; other applications that do not use the same lock could still create duplicate names.

# 4. Optional image handling

- A non-empty image URL must be HTTP or HTTPS. Responses must have an image/* content type and non-empty data.
- HTTP failures such as 404, URL/network errors, timeouts, invalid URLs, non-image responses, and empty responses are logged as warnings and treated as an image skip. The hostel and its other data continue to be inserted.
- When image download/validation fails, neither image_ids nor images rows are inserted for that hostel.
- When valid, the image bytes are Fernet-encrypted before being stored. IMAGE_ENCRYPTION_KEY is read from the environment when supplied; otherwise a new key is generated and must be persisted by the operator to decrypt those images later.

# 5. CSV fields and mapping

| CSV field | Database / handling |
|---|---|
| contact | Normalize to digits and remove a leading 91 only from a 12-digit number; use for duplicate check and hostel.contact. |
| name, type | hostel.name, hostel.type |
| price_starts, price_ends | Parsed as numeric values and inserted into hostel. |
| house_no, street, area, town, city, state, country, pincode | Inserted into address; pincode maps to address.zipcode. |
| location (first column) | Preserved as location by the CSV reader. |
| location (second duplicate column) | Preserved as location_2; preferred for hostel.location by the current implementation. |
| amenities | Comma-separated amenity names; inserted/reused in amenities and linked through hostel_amenities. |
| image_url | Optional URL; creates image records only after successful image download and validation. |

# 6. Failure outputs and operational notes

- Failed rows are written to <input-stem>_failed_records.csv and, when pandas/openpyxl are available, <input-stem>_processed_with_failures.xlsx.
- Invalid image URLs are warnings, not row failures. A row can count as inserted successfully even when its image is skipped.
- The process logs progress and total elapsed time. The code does not create or alter database tables, columns, indexes, or constraints.

## Implementation source of truth

Use the current hostel-csv-parser.py implementation for executable details. The original PDF includes older SQL DDL and source excerpts; in particular, the earlier image_ids auto-increment example and command-line invocation are superseded by this update.